Amazon SQS — Console Dashboard Wise Important Interview Questions & Answers

SQS
│
├── Queues
│ └── Individual Queue
│ ├── Details
│ ├── Configuration
│ │ ├── Visibility Timeout
│ │ ├── Message Retention Period
│ │ ├── Delivery Delay
│ │ ├── Maximum Message Size
│ │ └── Receive Message Wait Time
│ │
│ ├── Encryption
│ ├── Access Policy
│ ├── Dead-Letter Queue
│ ├── Redrive Policy
│ └── Monitoring
│
├── Dead-letter Queues
└── Settings

1. What is Amazon SQS?
Answer:
Amazon SQS stands for Simple Queue Service and is used to send and store messages between applications.
It helps separate the sender application from the receiver application so both do not need to work at the same time.
SQS is commonly used for asynchronous processing and improving application reliability.
For example:
Suppose users upload orders to our application.
The application puts each order into an SQS queue, and a worker application processes the orders one by one.
Example Diagram:
User
↓
Application
↓
SQS Queue
↓
Worker / EC2 / Lambda
↓
Process Order
Simple ga gurthupettuko:
SQS = Store messages until an application processes them

---

2. What is an SQS Queue?
Answer:
An SQS Queue is a temporary storage area where messages wait until a consumer processes them.
Producers send messages to the queue, and consumers receive messages from the queue.
The queue helps applications communicate asynchronously.
For example:
Suppose our application receives 1,000 orders at the same time.
Instead of processing everything immediately, the application puts the orders into an SQS queue and workers process them gradually.
Example Diagram:
Producer
↓
SQS Queue
↓
Consumer
Simple ga gurthupettuko:
Queue = Place where messages wait

---

3. What is the difference between Standard Queue and FIFO Queue?
Answer:
A Standard Queue provides very high throughput but does not guarantee strict message ordering.
A FIFO Queue preserves the order of messages and provides stronger duplicate-handling guarantees.
FIFO is useful when message order is important for the application.
For example:
If an application receives Order Created and then Order Cancelled, the order may be important.
We can use a FIFO queue when these messages must be processed in the correct order.
Example Diagram:

Standard:
Message 1 → Message 3 → Message 2

FIFO:
Message 1 → Message 2 → Message 3
Simple ga gurthupettuko:
Standard = High throughput
FIFO = Order matters

---

4. What is Visibility Timeout?
Answer:
Visibility Timeout is the amount of time a message becomes temporarily invisible after a consumer receives it.
This prevents another consumer from processing the same message while the first consumer is working on it.
If the consumer does not successfully process the message within the timeout, it can become visible again.
For example:
Suppose a worker receives an order and the visibility timeout is 60 seconds.
The message is hidden for 60 seconds while the worker processes the order.
Example Diagram:
SQS Message
↓
Consumer Receives
↓
Invisible for 60 sec
↓
Processing
↓
Delete Message
Simple ga gurthupettuko:
Visibility Timeout = Temporary hiding of a received message

---

5. What is Message Retention Period?
Answer:
Message Retention Period defines how long SQS keeps a message in the queue before automatically deleting it.
It is useful when consumers may be temporarily unavailable.
The retention period should be selected based on how long the application may need to process pending messages.
For example:
Suppose our worker application is down for some time.
SQS keeps the messages during the configured retention period so the worker can process them after it comes back.
Example Diagram:
Producer
↓
SQS Queue
↓
Message Waiting
↓
Consumer Down
↓
Consumer Comes Back
↓
Process Message
Simple ga gurthupettuko:
Retention = How long SQS keeps the message

---

6. What is Delivery Delay?
Answer:
Delivery Delay specifies how long SQS waits before making a newly sent message available to consumers.
It is useful when we intentionally want to delay message processing.
The delay applies to newly added messages.
For example:
Suppose we want an order-processing message to become available only after 30 seconds.
We can configure a delivery delay so consumers cannot receive that message immediately.
Example Diagram:
Producer
↓
SQS
↓
30 sec Delay
↓
Message Available
↓
Consumer

Simple ga gurthupettuko:
Delivery Delay = Delay before a new message becomes available

---

7. What is Receive Message Wait Time?
Answer:
Receive Message Wait Time controls how long SQS waits for messages when a consumer requests messages.
This is related to **long polling** and can reduce unnecessary empty responses.
It can help reduce API calls and improve the efficiency of consumers.
For example:
Suppose our consumer asks SQS for a message but the queue is temporarily empty.
With long polling, SQS can wait for a message instead of immediately returning an empty response.
Example Diagram:
Consumer
↓
Receive Message
↓
SQS waits
↓
Message arrives
↓
Consumer receives it
Simple ga gurthupettuko:
Long Polling = Wait for a message instead of immediately returning empty

---

8. What is a Dead-Letter Queue (DLQ)?
Answer:
A Dead-Letter Queue is a separate SQS queue used to store messages that could not be processed successfully.
It helps us isolate failed messages instead of repeatedly processing them in the main queue.
We can later investigate and reprocess those failed messages.
For example:
Suppose an order message fails processing several times because of invalid data.
After the configured retry limit, SQS can move that message to the Dead-Letter Queue for investigation.
Example Diagram:
Main Queue
↓
Consumer
↓
Processing Failed
↓
Retry
↓
Failed Again
↓
Dead-Letter Queue
Simple ga gurthupettuko:
DLQ = Store messages that repeatedly fail

---

9. What is Redrive Policy?
Answer:
A Redrive Policy defines when messages should be moved from the main queue to a Dead-Letter Queue.
It uses a setting called **Maximum Receives**, which determines how many times a message can be received before moving to the DLQ.
This prevents continuously failing messages from staying in the main queue.
For example:
Suppose Maximum Receives is configured as 3.
If a message is received and fails processing repeatedly, after the configured limit it can be moved to the DLQ.
Example Diagram:
Main Queue
↓
Receive 1 → Fail
↓
Receive 2 → Fail
↓
Receive 3 → Fail
↓
DLQ
Simple ga gurthupettuko:
Redrive Policy = Decide when failed messages move to DLQ

---

10. What is SQS Encryption?

Answer:
SQS encryption protects messages stored in the queue from unauthorized access to the stored data.
SQS supports server-side encryption using AWS-owned or AWS KMS keys, depending on the configuration.
Encryption is especially important when the messages contain sensitive application information.
For example:
Suppose our queue contains customer payment-related information.
We can enable server-side encryption so messages stored in SQS are protected.
Example Diagram:
Application
↓
Encrypted Message
↓
SQS Queue
↓
Protected Data
Simple ga gurthupettuko:
SQS Encryption = Protect messages stored in the queue

---

11. What is an SQS Access Policy?
Answer:
An SQS Access Policy controls which AWS principals or services can access the queue and what actions they can perform.
It is a resource-based policy attached to the SQS queue.
We can use it to allow or restrict actions such as sending and receiving messages.
For example:
Suppose an application in another AWS account needs to send messages to our queue.
We can configure the queue policy to allow that account to perform the required SQS action.
Example Diagram:
Application / AWS Account
↓
Queue Policy
↓
SQS Queue
↓
Allow / Deny
Simple ga gurthupettuko:
Access Policy = Who can access the SQS queue

---

12. What is SQS Monitoring?
Answer:
SQS monitoring helps us understand the health and activity of a queue using CloudWatch metrics.
Important metrics include the number of visible messages, messages being processed, and messages delayed.
These metrics help identify processing delays and growing backlogs.
For example:
Suppose our application receives messages faster than workers can process them.
The number of messages in the queue increases, which tells us that we may need more consumers or better processing capacity.
Example Diagram:
Producer
↓
SQS Queue
↓
CloudWatch Metrics
↓
Message Count
↓
Monitor / Alert
Simple ga gurthupettuko:
SQS Monitoring = Watch queue messages and processing health

---

13. What is the difference between SQS and SNS?
Answer:
SQS is a message queue where messages wait until a consumer processes them.
SNS is a publish/subscribe service that sends messages to multiple subscribers.
SQS is commonly used for asynchronous processing, while SNS is commonly used for notifications and fan-out.
For example:
Suppose an order is created and multiple systems need to know about it.
SNS can publish the event, and multiple SQS queues can receive copies for different applications.
Example Diagram:
SNS
/ \
↓ ↓
SQS-A SQS-B

↓ ↓
App-A App-B
Simple ga gurthupettuko:
SQS = Queue
SNS = Publish / Subscribe

---

14. What happens if a consumer receives a message but does not delete it?
Answer:
After a consumer receives a message, SQS makes it invisible for the configured Visibility Timeout.
If the consumer does not delete the message before the timeout expires, the message can become visible again.
Another consumer may then receive and process the message again.
For example:
Suppose a worker receives a message but crashes before completing the task.
After the visibility timeout expires, SQS makes the message available again so another worker can process it.
Example Diagram:
SQS
↓
Consumer receives
↓
Message invisible
↓
Consumer crashes
↓
Visibility Timeout expires
↓
Message visible again
Simple ga gurthupettuko:
No Delete + Timeout = Message can be received again

---

15. What is the basic SQS message flow?
Answer:
A producer sends a message to an SQS queue, and the message waits until a consumer receives it.
The consumer processes the message and normally deletes it after successful processing.
If processing fails, the message can become visible again and eventually move to a DLQ based on the configuration.
For example:
Suppose our website receives customer orders.
The website sends orders to SQS, a worker processes them, and successfully processed messages are deleted from the queue.
Example Diagram:
Producer
↓
SQS Queue
↓
Consumer
↓
Process Message
↓
Delete Message
Simple ga gurthupettuko:
Send → Queue → Receive → Process → Delete
ChatGPT is AI and can make mistakes.
Chat with ChatGPT